

# Biểu diễn số thực trong máy tính

Quách Lê Nhật Huy - 24122016

## 1 Giới thiệu vấn đề:

Tất cả dữ liệu trong máy tính, ở tầng thấp nhất, đều được biểu diễn dưới dạng các chuỗi bit 0 và 1. Đối với các dữ liệu dạng số như số dương (unsigned) hoặc số nguyên (integer), ta có thể dùng hệ nhị phân thông thường hoặc dạng bù hai đối với số nguyên âm.

Tuy nhiên, đối với số thực, do có chứa phần thập phân, có thể chứa các giá trị rất dài và khó biểu diễn mặc dù không nhất thiết có giá trị toán học lớn (ví dụ:  $1/3 = 0.3333\dots$ ). Sau cùng, bản chất của số thực là một giá trị liên tục, vô hạn, trong khi bộ nhớ của máy tính mang tính hữu hạn, rời rạc.

Chính vì vậy, việc biểu diễn số thực trong máy tính đặt ra nhiều khó khăn và là một vấn đề rất lớn trong lĩnh vực công nghệ và khoa học máy tính. Những phương pháp tối ưu nhất hiện nay tập trung vào việc cân bằng giữa tốc độ tính toán, bộ nhớ và độ chính xác khi biểu diễn số thực.

## 2 Cách biểu diễn phần thập phân:

Cách biểu diễn phần thập phân cơ bản cũng tương tự như phần nguyên, tuân theo nguyên tắc: cứ sang phải 1 vị trí thì số mũ giảm 1 đơn vị, với bit đầu tiên sau dấu phẩy ứng với số mũ  $-1$ .

$$\text{Ví dụ: } 1101.011_2 = 2^3 + 2^2 + 2^0 + 2^{-2} + 2^{-3} = 13.375_{10}$$

### 3 Dấu chấm tĩnh và dấu chấm động:

Vấn đề lớn nhất ở số thực nằm ở tính liên tục của số thực, cụ thể hơn là việc phân bổ bộ nhớ để biểu diễn phần thập phân một cách chính xác và hiệu quả. Ta có 2 phương pháp chính là **dấu chấm tĩnh** và **dấu chấm động**.

#### 3.1 Dấu chấm tĩnh (fixed-point representation):

##### 3.1.1 Nguyên lý hoạt động:

Số bit dùng để biểu diễn phần nguyên và thập phân được quy định trước và không thay đổi.

Ví dụ: Dùng 8 bit để lưu trữ số thực, trong đó quy ước 4 bit đầu biểu diễn phần nguyên, 4 bit sau biểu diễn phần thập phân. Như vậy số  $10101100_2$  được hiểu là  $1010.1100_2$ .

##### 3.1.2 Ưu điểm:

Thiết kế đơn giản, tốc độ xử lý nhanh, có thể thực hiện phép cộng, trừ tương tự như một số nguyên.

##### 3.1.3 Nhược điểm:

Phân bổ tài nguyên không hợp lý và có thể gây lãng phí. Vì dấu chấm cố định, các bit bị lãng phí khi số thực có phần nguyên và phần thập phân không đồng đều (một phần rắc rối cần nhiều bit biểu diễn, phần còn lại đơn giản cần ít bit biểu diễn). Điều này cũng đặt giới hạn lớn lên dải giá trị có thể biểu diễn so với số bit đang dùng.

## 3.2 Dấu chấm động (floating-point representation):

### 3.2.1 Nguyên lý hoạt động:

Vị trí dấu chấm có thể thay đổi tùy vào số thực đang biểu diễn, cho phép phân bổ linh hoạt số bit biểu diễn kể cả khi phần nguyên và phần thập phân không đồng đều.

Cấu trúc tổng quát của dãy bit bao gồm phần **dấu (sign)**, **mũ (exponent)** và **định trị (mantissa)**:

- **Phần dấu (sign):** Thường chỉ chiếm 1 bit đầu tiên, xác định dấu của giá trị.
- **Phần mũ (exponent):** Xác định vị trí của dấu phẩy. Bằng việc thay đổi phần mũ, dấu phẩy có thể dịch chuyển sang trái (dành nhiều bit cho phần thập phân, giá trị số thực nhỏ) hay sang phải (dành nhiều bit cho phần nguyên, giá trị số thực lớn). Tính bằng phương pháp Bias (Excess-k)
- **Phần định trị (mantissa):** Chứa các chữ số của số thực, quyết định độ chính xác của giá trị được biểu diễn.

Ví dụ: Dùng 8 bit để lưu trữ số thực, trong đó có 1 bit dấu, 3 bit mũ, 4 bit định trị. Ta quy ước biểu diễn số  $-2.5$  như sau:

- Dấu: ta quy ước số 1 cho số âm, số 0 cho số dương. Vì  $-2.5$  là số âm nên bit dấu là 1.
- Chuyển  $2.5$  sang nhị phân, ta có phần nguyên (2) là  $10$  và phần thập phân (0.5) là  $.1$ . Như vậy  $2.5_{10} = 10.1_2$ . Ta thêm số 0 ở cuối để đủ 4 bit định trị:  $1010$ .
- Xác định số mũ, ta quy ước số mũ biểu diễn số chữ số cần dịch dấu chấm về bên phải. Vì  $10.1_2$  dịch dấu chấm về bên phải 2 chữ số, phần mũ là  $2_{10} = 110_2$  (theo Excess-k với  $k = 4$ ).
- Kết quả cuối cùng:  $-2.5_{10} = 1\ 110\ 1010_2$

### 3.2.2 Ưu điểm:

Phân bổ tài nguyên linh động, cho phép biểu diễn chính xác phần thập phân khi số thực có giá trị nhỏ (phần nguyên nhỏ) và dành nhiều bit hơn cho phần nguyên khi giá trị của số thực lớn. Điều này cũng làm dải biểu diễn cực kỳ rộng với số bit sử dụng.

Nhờ được chuẩn hóa bằng **IEEE-754**, dấu chấm động hiện là tiêu chuẩn toàn cầu cho việc biểu diễn số thực trên hầu kiến trúc, từ chip Intel trên PC đến chip ARM trên điện thoại.

### 3.2.3 Nhược điểm:

- **Sai số làm tròn:** Như đã nói trên, vì bộ nhớ máy tính hữu hạn, rời rạc, trong khi số thực vô hạn, liên tục, máy tính buộc phải làm tròn phần thập phân khi nó quá phức tạp, vô hạn tuần hoàn hoặc số thực là số vô tỉ...dẫn đến sai số.
- **Chi phí tính toán:** Vì thiết kế dấu chấm động phức tạp hơn nhiều so với dấu chấm tĩnh, việc thực hiện các phép cộng, trừ, nhân, chia cũng đòi hỏi nhiều tài nguyên hơn và làm giảm hiệu suất nghiêm trọng, đặc biệt trên các CPU giá rẻ.
- **Hiện tượng "nuốt số":** Vì dấu chấm di động linh hoạt dựa trên sự phức tạp của phần nguyên và phần thập phân. Khi ta thực hiện cộng một số thực có phần nguyên cực lớn với một số thực có phần nguyên và phần thập phân cực nhỏ, số nhỏ có thể sẽ bị "nuốt" hoàn toàn (ví dụ:  $100000000.0 + 0.000000001 = 100000000.0$ ).

## 4 Tiêu chuẩn IEEE-754:

### 4.1 Giới thiệu:

Như ví dụ 8-bit ở trên, với cùng một cấu trúc tổng quát, ta có thể tự quy ước cách chia bit cho mỗi phần và cách tính giá trị khi biểu diễn số thực. Để thống nhất cách biểu diễn và tính toán với số thực bằng hệ nhị phân trên mọi nền tảng, năm 1985, chuẩn IEEE-754 đã ra đời.

Tiêu chuẩn IEEE-754 hiện nay là tiêu chuẩn toàn cầu ở gần như toàn bộ các kiến trúc hay hệ thống cần lưu, biểu diễn và tính toán số thực.

### 4.2 Định dạng phổ biến:

IEEE-754 quy định hai kích thước tiêu chuẩn:

- **Single precision (32-bit):** Biểu diễn bằng 32 bit, gồm 1 bit dấu, 8 bit mũ (Bias là 127), 23 bit định trị.
- **Double precision (64-bit):** Biểu diễn bằng 64 bit, gồm 1 bit dấu, 11 bit mũ (Bias là 1023), 52 bit định trị.
- Ngoài ra còn các kích thước khác như **Half precision (16-bit)** hay **Quadruple precision (128-bit)**...

### 4.3 Nguyên lý hoạt động:

#### 4.3.1 Công thức tổng quát:

Dãy bit luôn được chia làm 3 phần nối tiếp nhau từ trái sang phải: **dấu (S)**, **mũ (E)**, **định trị (M)**.

Giá trị theo hệ thập phân của số thực được tính từ dãy bit theo công thức sau:

$$V = (-1)^S \times 1.M \times 2^{E-Bias}$$

Trong đó:

- $V$ : Giá trị ở hệ thập phân.
- $S$ : Bit dấu, 0 cho số dương và 1 cho số âm.
- $1.M$ : Phần định trị ghép thêm một "bit ẩn" 1.
- $E$ : Giá trị phần mũ, được tính theo phương pháp thông thường (tương tự như cho số nguyên dương).
- $Bias$ : Độ lệch, mang giá trị 127 đối với định dạng 32-bit và 1023 đối với định dạng 64-bit.

Ví dụ:

- Chuyển từ  $0\ 10000000\ 11000000000000000000000_2$  (32-bit) sang thập phân:

$$- S = 0$$

$$- E = 10000000_2 = 128_{10}$$

$$- 1.M = 1.11_2 = 1.75_{10}$$

$$\implies V = (-1)^0 \times 1.75 \times 2^{128-127} = 3.5_{10}$$

- Chuyển từ  $-0.75_{10}$  (32-bit) sang nhị phân:

$$- \text{Số âm} \implies S = 1$$

$$- 0.75_{10} = 0.11_2 = 1.1_2 \times 2^{-1}$$

$$\implies M = 100000000000000000000000$$

$$\implies E = -1 + 127 = 126_{10} = 01111110_2$$

$$\implies \text{Dãy bit là } \langle SEM \rangle = 1\ 01111110\ 100000000000000000000000.$$

### 4.3.2 Cơ chế độ lệch (Bias):

Vì số mũ được lưu theo nguyên tắc Excess- $k$  với độ lệch  $k$  biết trước, phần mũ lưu trong bộ nhớ luôn là một số nguyên dương. Việc này cực kỳ tiện lợi trong quá trình so sánh hai số thực: trừ bit dấu, CPU chỉ cần quét dãy bit từ trái qua phải, chuỗi bit nào có giá trị lớn hơn thì số thực đó có trị lớn hơn.

### 4.3.3 Cơ chế chuẩn hóa và bit ẩn:

Trong hệ nhị phân, chỉ có hai chữ số là 0 và 1. Bất kỳ giá trị nào khác 0 đều có ít nhất một chữ số 1 trong dãy bit.

Vì thế, mọi số nhị phân được dịch chuyển đến khi chỉ còn đúng một chữ số 1 nằm bên trái dấu phẩy, đây gọi là cơ chế **chuẩn hóa**. Sau khi chuẩn hóa xong, vì biết giá trị mang định dạng  $1.xxxx\dots$ , IEEE-754 không lưu số 1 này để tiết kiệm bộ nhớ và tăng độ chính xác với cùng tài nguyên phần cứng, đây là cơ chế **bit ẩn**.

Chính vì vậy, từ công thức tổng quát ta thấy dấu chấm được di chuyển  $E - Bias$  lần từ vị trí bit 1 đó chứ không phải từ đầu.

Vì cơ chế này, số 0 được biểu diễn bằng quy ước đặc biệt.

### 4.3.4 Giới hạn dãy giá trị biểu diễn:

- Single precision (32-bit):

- Giá trị tuyệt đối lớn nhất:  $0\ 11111110\ 111111111111111111111111_2 = (2 - 2^{-23}) \times 2^{127} \approx 3.40282347 \times 10^{38}$

- Giá trị tuyệt đối nhỏ nhất:  $0\ 00000001\ 000000000000000000000000_2 = 1.0 \times 2^{-126} \approx 1.17549435 \times 10^{-38}$

- Double precision (64-bit):

- Giá trị tuyệt đối lớn nhất:  $0\ 111111111110\ 111\dots111_2 = (2 - 2^{-52}) \times 2^{1023} \approx 1.79769313 \times 10^{308}$

- Giá trị tuyệt đối nhỏ nhất:  $0\ 00000000001\ 000...000_2 = 1.0 \times 2^{-1022} \approx 2.22507386 \times 10^{-308}$

#### 4.3.5 Các giá trị đặc biệt:

##### 1. Số dưới chuẩn (Subnormal/Denormalized numbers):

- Khi tính toán với số thực, nếu số được tính ra quá nhỏ, có thể vô tình bị **Underflow** và kết quả làm tròn xuống 0, dẫn đến mất số đột ngột.
- Để khắc phục vấn đề này, những số "không chuẩn hóa" được tạo ra: khi phần mũ được lấp đầy số 0. Phần định trị sẽ được hiểu là  $0.xxxx...$  thay vì  $1.xxxx...$  và số mũ mang giá trị ở mức thấp nhất ( $-126$  cho 32-bit và  $-1022$  cho 64-bit).
- Điều này làm cho "giá trị nhỏ nhất thực sự" của biểu diễn số thực là  $0...001$ , tương đương  $2^{-149}$  cho 32-bit và  $2^{-324}$  cho 64-bit.

##### 2. Số 0:

- Từ giá trị siêu nhỏ, ta thấy cơ chế biểu diễn số 0 cũng bao gồm việc đặt tất cả phần mũ, và bên cạnh đó phần định trị, về 0.
- Vì cách hoạt động của IEEE-754, có 2 dãy nhị phân cho số 0 là  $100...0$ , tương ứng với  $-0$  và  $000...0$ , tương ứng với  $+0$ .
- Trong các ngôn ngữ lập trình phổ biến (như C++, Java, Python), khi so sánh  $+0.0 == -0.0$  kết quả vẫn sẽ trả về *True*. Tuy nhiên nếu lấy  $1/(+0.0)$  thì sẽ ra  $+\infty$  còn  $1/(-0.0)$  thì sẽ ra  $-\infty$ .

##### 3. Vô cực:

- Là kết quả của các hiện tượng **tràn số (Overflow)** hoặc phép chia cho 0.
- Được biểu diễn bằng phần mũ toàn số 1, phần định trị toàn số 0, phần dấu quy định dấu của vô cực.



- Trong các ngôn ngữ lập trình phổ biến, vô cực vẫn có thể tham gia vào phép toán. Ví dụ:  $\infty + 1 = \infty$ ,  $1/\infty = 0$ ...

#### 4. Giá trị không phải số (NaN - Not a Number):

- Là kết quả của các phép toán vô lý hoặc lỗi logic. Ví dụ như  $0.0/0.0$ ,  $\infty - \infty$ ,  $\infty/\infty$ ...
- Được biểu diễn bằng phần mũ toàn số 1 và phần định trị có ít nhất một bit khác 0.
- Trong các ngôn ngữ lập trình phổ biến, giá trị *NaN* không bằng bất cứ giá trị nào, kể cả chính nó ( $NaN == NaN$  trả về *False*).

## 5 Mở rộng:

Dù tiêu chuẩn IEEE-754 là tiêu chuẩn hiện đại tối ưu việc biểu diễn và tính toán số thực, song vẫn không tránh được sai số khi làm tròn số thực (hiện tượng phổ biến  $0.1 + 0.2 = 0.30000000000000004$  trong lập trình).

Trong những trường hợp đòi hỏi độ chính xác hoàn hảo, có một số phương pháp khác có thể đổi được hiệu suất và tốc độ tính toán cho độ chính xác mong muốn.

### 5.1 Biến tất cả thành số nguyên (Scaled Integer):

- **Nguyên lý hoạt động:** Thay vì lưu giá trị dưới dạng số thực, nếu có thể xác định trước độ chính xác cần mong muốn (ví dụ: 3 chữ số thập phân), ta có thể nhân mọi số với 1000 (scale - tỉ lệ nhân) rồi lưu dưới dạng số nguyên.
- **Ứng dụng:** Thường được ứng dụng cho các ngân hàng, game engines, hệ thống nhúng (embedded systems)...
- **Tính chất:** Đạt được độ chính xác mong muốn, kết quả ra luôn luôn giống nhau bất kể được tính toán trên hệ thống nào.

## 5.2 Độ chính xác vô hạn (Arbitrary-precision Arithmetic):

- **Nguyên lý hoạt động:** Lưu giá trị số dưới dạng một mảng (array) hoặc chuỗi (string) chứa từng chữ số.
- **Ứng dụng:** Thường được ứng dụng cho khoa học vũ trụ (tính quỹ đạo vệ tinh), mật mã học (cryptography), tính  $\pi$ ...
- **Tính chất:** Không có giới hạn độ lớn hay sai số, bù lại xử lý cực kỳ chậm, các phép toán cộng, trừ, nhân, chia được mô phỏng lại bằng phần mềm.

## 5.3 Biểu diễn dạng phân số (Symbolic Computation):

- **Nguyên lý hoạt động:** Lưu số thực dưới dạng 2 số nguyên, đóng vai trò tử số và mẫu số của một phân số.
- **Ứng dụng:** Thường được ứng dụng cho các phần mềm toán học như Mathematica, MATLAB,...
- **Tính chất:** Không ứng dụng được cho các số vô tỉ.

## 5.4 Dấu chấm động hệ thập phân (Decimal Floating-point):

- **Nguyên lý hoạt động:** Được update từ IEEE-754, dùng hệ cơ số 10 ở tầng phần cứng, giúp biểu diễn chính xác số thực mà không sinh ra sai số vô hạn tuần hoàn.
- **Ứng dụng:** Thường được ứng dụng cho dữ liệu thương mại, thuế quan, doanh nghiệp...
- **Tính chất:** Đánh đổi tốc độ xử lý cho độ chính xác.

## 6 Kết luận:

Biểu diễn số thực trong máy tính là một bài toán phức tạp bởi bản chất liên tục và vô hạn của nó. Hiện nay, tiêu chuẩn IEEE-754 là tiêu chuẩn toàn cầu, được sử dụng bởi hầu hết các chương trình, hệ thống trên thế giới để lưu trữ, biểu diễn và tính toán số thực.

Tiêu chuẩn IEEE-754 là tiêu chuẩn sử dụng dấu chấm động, tối ưu, linh hoạt cân bằng giữa độ chính xác và quy mô biểu diễn dựa trên độ lớn của số thực. Tiêu chuẩn IEEE-754 có dải giá trị biểu diễn rộng và hỗ trợ những giá trị đặc biệt như vô cực và NaN. Dù vậy, tiêu chuẩn vẫn gặp phải vấn đề sai số khi làm tròn cần được khắc phục.

Bên cạnh tiêu chuẩn IEEE-754, các phương pháp khác như độ chính xác vô hạn, biểu diễn số thực bằng số nguyên hay dấu chấm động hệ thập phân...cũng được tạo ra nhằm đánh đổi tốc độ và hiệu suất tính toán để khắc phục điểm yếu của IEEE-754, đạt được độ chính xác mong muốn.